IntelliCode Review: AI-Powered Code Review and Quality Assurance System

CS 791 – Software Engineering

Safaa Patel

Department of Computer Science and Engineering

College of Engineering

University of Nevada, Reno

Project P1: Concept

Instructor: Dr. Sergiu Dascalu

November 1, 2025

Contents

1 Concept **3**
- 1.1 Main Goals of the Project . . . 3
- 1.2 Main Functionality and Characteristics . . . 3
- 1.3 Intended Technology . . . 4
- 1.4 Intended Users and Key Usability Goals . . . 4
- 1.5 Notes on Similar/Related Products . . . 5
- 1.6 Novelty of Your Solution . . . 5
- 1.7 Professional Development Impact . . . 6
- 1.8 Author Bio . . . 6

2 Requirements **7**
- 2.1 Requirements Elicitation: Interview Findings . . . 7
  - 2.1.1 Interview Questions and Summary of Findings . . . 7
- 2.2 Software Requirements . . . 8
  - 2.2.1 Functional Requirements . . . 9
  - 2.2.2 Non-Functional Requirements . . . 11

3 Use Case Modeling **12**
- 3.1 Use Case Diagram . . . 12
- 3.2 Detailed Use Cases . . . 13
  - 3.2.1 Expanded Use Case Templates . . . 15

4 Glossary of Terms **19**

5 References **21**

Abstract

IntelliCode Review is an intelligent code review platform that leverages machine learning and static analysis to automate software quality assurance processes. The system addresses the time-intensive nature of manual code reviews by providing automated detection of code smells, security vulnerabilities, and adherence to coding standards. The significance lies in reducing code review time by 60% while maintaining comprehensive quality checks. The implementation utilizes Python with FastAPI backend, React frontend, and transformer-based models for intelligent code analysis. This project benefits software development teams by enabling faster deployment cycles, consistent code quality enforcement, and knowledge transfer through automated explanations. The deliverable includes a web-based review interface, CI/CD integration capabilities, and a machine learning model trained on open-source repositories.

1 Concept

1.1 Main Goals of the Project

IntelliCode Review aims to revolutionize the software code review process by integrating artificial intelligence with traditional static analysis tools. The primary goal is to create an intelligent assistant that can perform preliminary code reviews, identify potential issues, suggest improvements, and learn from human reviewer feedback to continuously improve its recommendations. This system will serve as a force multiplier for development teams, allowing human reviewers to focus on architectural decisions and complex logic while the AI handles routine quality checks, style enforcement, and common vulnerability detection. The project addresses a critical bottleneck in modern software development: the code review process. Studies show that developers spend 4-6 hours weekly on code reviews, with 40 percent of that time spent on issues that could be automatically detected. IntelliCode Review will reduce this burden while maintaining the collaborative and educational aspects of code review through its intelligent commenting and explanation features.

1.2 Main Functionality and Characteristics

Core Features:

- **Intelligent Code Analysis**: Utilizes transformer-based models (CodeBERT) fine-tuned on code review datasets to understand code semantics beyond simple pattern matching
- **Multi-Language Support**: Initial support for Python, JavaScript, and Java with extensible architecture for additional languages
- **Security Vulnerability Detection**: Integration with OWASP dependency checker and custom ML models trained on CVE databases
- **Code Smell Detection**: Identifies 25+ code smells including long methods, god classes, duplicate code, and inappropriate intimacy
- **Style Compliance**: Enforces team-specific coding standards with configurable rule sets
- **Contextual Suggestions**: Provides explanations and refactoring suggestions with code examples
- **Learning System**: Incorporates human reviewer feedback to improve future recommendations
- **CI/CD Integration**: Seamless integration with GitHub, GitLab, and Bitbucket through webhooks
- **Review Dashboard**: Comprehensive analytics showing code quality trends, common issues, and team performance metrics
- **Collaborative Features**: Threaded discussions on AI-identified issues with @mentions and resolution tracking

Key Characteristics:

- Real-time analysis with sub-3-second response for files under 1000 lines

- 85
- Privacy-focused with on-premise deployment option
- RESTful API for custom integrations
- Responsive web interface supporting desktop and tablet devices

1.3 Intended Technology

- **Backend Platform**: Linux-based cloud infrastructure (AWS/Azure) with containerization
- **Primary Language**: Python 3.11+
- **Backend Framework**: FastAPI for high-performance async API endpoints
- **Frontend**: React 18 with TypeScript for type safety
- **Machine Learning**: PyTorch 2.0 for model training and inference, CodeBERT transformer model (Microsoft), Scikit-learn for classical ML components
- **Static Analysis Tools**: Pylint, Flake8, Bandit (Python), ESLint, SonarQube (JavaScript), PMD, SpotBugs (Java)
- **Database**: PostgreSQL 15 for relational data, Redis for caching
- **Message Queue**: RabbitMQ for asynchronous task processing
- **Version Control Integration**: GitHub API, GitLab API, Bitbucket REST API
- **Containerization**: Docker and Kubernetes for orchestration
- **Testing**: Pytest, Jest, Selenium for end-to-end testing
- **CI/CD**: GitHub Actions for automated testing and deployment
- **Monitoring**: Prometheus and Grafana for system metrics

1.4 Intended Users and Key Usability Goals

Primary Users:

- Software developers submitting pull requests for review
- Senior developers and tech leads performing code reviews
- DevOps engineers integrating quality gates into CI/CD pipelines

Secondary Users:

- Engineering managers monitoring code quality metrics
- New team members learning coding standards

Key Usability Goals:

1. **Efficiency**: Reduce average code review time from 45 minutes to 18 minutes per pull request

2. **Learnability**: Enable new users to submit their first code for review within 5 minutes of account creation
3. **Accuracy**: Achieve less than 15
4. **Transparency**: Provide clear explanations for every flagged issue with severity ratings
5. **Integration**: Zero-friction integration with existing workflows through Git platform webhooks

User Benefits:

- Developers receive instant feedback reducing context-switching delays
- Teams maintain consistent code quality standards across all contributors
- Junior developers learn best practices through detailed explanations
- Organizations reduce technical debt accumulation by 40
- Faster onboarding for new team members through standardized feedback

1.5 Notes on Similar/Related Products

1. SonarQube [1]: Open-source platform for continuous code quality inspection. Strengths include comprehensive language support and detailed metrics. However, it lacks AI-powered contextual suggestions and requires significant configuration for optimal results.

2. CodeClimate [2]: Commercial static analysis platform with technical debt tracking. Provides good quality metrics but limited in intelligent refactoring suggestions and doesn't learn from team-specific patterns.

3. DeepCode (Snyk Code) [3]: AI-powered code review tool using machine learning. Strong in security vulnerability detection but limited in explaining its reasoning and doesn't support collaborative review features.

4. GitHub Copilot [4]: AI pair programmer that suggests code completions. While excellent for code generation, it doesn't provide comprehensive review capabilities or quality assurance features.

5. Codacy [5]: Automated code review platform with CI/CD integration. Good coverage of standard issues but lacks the learning capability and contextual intelligence of ML-based approaches.

1.6 Novelty of Your Solution

IntelliCode Review introduces several innovative characteristics that differentiate it from existing solutions:

1. Hybrid Intelligence Architecture: Combines traditional rule-based static analysis with transformer-based deep learning models, leveraging the reliability of established tools with the contextual understanding of modern AI.

2. Adaptive Learning System: Implements a feedback loop where human reviewer decisions (accepting/rejecting AI suggestions) continuously refine the model's recommendations, creating team-specific intelligence that understands organizational preferences.

3. Explanation Generation: Uses natural language generation to provide detailed explanations for each identified issue, including severity reasoning, impact analysis, and concrete refactoring examples—going beyond simple rule violation notifications.

4. Proactive Security Intelligence: Integrates real-time CVE database monitoring with semantic code analysis to identify potential vulnerabilities before they're officially documented, using pattern recognition from historical security issues.

5. Developer Experience Focus: Prioritizes reducing review fatigue through intelligent issue prioritization, consolidated reporting, and seamless integration rather than overwhelming developers with exhaustive reports.

1.7 Professional Development Impact

This project directly aligns with my career objective of becoming a technical lead specializing in developer productivity and AI-enhanced software engineering tools. The development of IntelliCode Review provides hands-on experience with several critical technologies:

1. Technical Expertise: Deepens knowledge in machine learning model deployment, large-scale system architecture, and real-time data processing—skills highly valued in senior engineering roles.

2. AI/ML Specialization: Provides practical experience in applying transformer models to software engineering problems, positioning me for roles in AI-assisted development tools, a rapidly growing sector.

3. Full-Stack Capabilities: Develops end-to-end product development skills from ML model training to production deployment, essential for technical leadership positions.

4. Domain Knowledge: Builds expertise in software quality assurance and DevOps practices, complementing my computer science foundation with practical software engineering principles.

1.8 Author Bio

Figure 1: Author

Safaa Patel is a graduate student in Computer Science and Engineering at the University of Nevada, Reno. She holds a B.Tech in Computer Science (Data Science specialization) from D. Y. Patil International University, India. Her interests include data science, AI-assisted software development, and computational workflows. She has prior experience at Azulene Labs, Johnson Controls, and Chanakya Software Services, and co-authored the IEEE paper *"Sentiment Analysis of GitHub Commit Comments Using Machine Learning" (2024).*

2 Requirements

2.1 Requirements Elicitation: Interview Findings

Requirements elicitation was conducted through structured interviews with three software engineering professionals representing different experience levels and roles. The interviews focused on understanding current pain points in code review processes, desired features for automated review tools, and factors affecting tool adoption. Each stakeholder was asked the same ten questions to ensure consistency and enable comparative analysis across different perspectives.

2.1.1 Interview Questions and Summary of Findings

Q1: What are the most time-consuming aspects of code reviews in your current workflow?

Summary: All three stakeholders identified that checking for style inconsistencies, basic security vulnerabilities, and repetitive issues consumes 30-40% of review time. Senior developers spend additional time explaining the rationale behind feedback to junior developers. Managers noted that inconsistent review thoroughness across team members creates quality variations. The consensus is that automated detection of routine issues would free reviewers to focus on architecture, business logic, and knowledge transfer.

Q2: How do you currently handle code quality enforcement across your team?

Summary: Teams primarily use linting tools (ESLint, Pylint) combined with manual review checklists. However, stakeholders reported that these tools catch only surface-level issues. Junior developers often miss nuanced problems that automated tools don't detect, requiring multiple review iterations. Managers struggle with ensuring consistent application of standards across team members. The lack of intelligent, context-aware analysis is a common gap in current tooling.

Q3: What would make an AI code review tool trustworthy enough for your team to adopt?

Summary: Transparency emerged as the critical factor for trust. All stakeholders emphasized the need to understand *why* something is flagged, see concrete examples of better approaches, and have the ability to easily override false positives. A false positive rate above 20% would lead to tool abandonment. Senior developers want confidence intervals or certainty scores, while junior developers value educational explanations. Managers need audit trails showing what the AI detected versus what humans approved.

Q4: How often does your team discover bugs during code review versus after deployment?

Summary: Stakeholders reported that 55-65% of bugs are caught during review, but the remaining bugs often stem from edge cases, integration issues, or race conditions not obvious in isolated code changes. Senior developers noted that bugs escaping to production typically involve cross-module interactions or implicit assumptions. Junior developers admitted to occasionally missing security implications of seemingly benign changes. An intelligent tool understanding context beyond individual files could significantly improve bug detection rates.

Q5: What integration points would be essential for adoption in your organization?

Summary: GitHub integration was unanimously identified as mandatory, with GitLab as secondary priority. Stakeholders want seamless webhook-based automation requiring zero manual triggering. Senior developers requested Slack notifications for high-severity issues. Managers emphasized the need for dashboard analytics integrated with existing project management tools. API

endpoints for custom CI/CD pipeline integration were mentioned by DevOps-focused stakeholders. Mobile accessibility for reviewing on-the-go was desired but not critical.

Q6: How do you balance code review thoroughness with development velocity?

Summary: This represents a persistent tension across all experience levels. Teams typically limit reviews to 400 lines per PR and aim for 24-hour turnaround, but critical issues create bottlenecks. Senior developers reported that rushed reviews miss subtle issues, while overly thorough reviews slow feature delivery. Junior developers feel pressure to approve quickly even when uncertain. Managers track review velocity metrics but struggle to maintain consistent quality. An AI performing fast preliminary reviews could accelerate the process while maintaining thoroughness.

Q7: What coding standards vary most across different teams in your organization?

Summary: Naming conventions, error handling approaches, and comment density exhibit the greatest variation. Senior developers noted that each team evolves its own preferences, making cross-team collaboration challenging. Junior developers find it confusing when moving between projects with different standards. Managers identified this inconsistency as a source of friction and onboarding delays. The ability to configure team-specific rule sets while maintaining organizational baseline standards would address this challenge.

Q8: Have you used any AI-assisted development tools? What were the pros and cons?

Summary: Most stakeholders have experimented with GitHub Copilot or similar code generation tools. They praised the speed for boilerplate code but expressed concerns about security risks and inefficient patterns in AI suggestions. Senior developers emphasized that AI assistants should complement, not replace, human judgment. Junior developers valued the learning aspect but worried about becoming overly dependent. Managers want metrics demonstrating AI tool impact on productivity and quality. The consensus is that AI for *reviewing* code is more valuable than AI for *generating* code.

Q9: What metrics would demonstrate that a code review tool is providing value?

Summary: Stakeholders identified several key performance indicators: reduction in average review time, fewer bugs escaping to production, decreased time-to-merge for PRs, reduction in technical debt accumulation, and positive developer sentiment scores. Senior developers want to track false positive rates and AI suggestion acceptance rates. Junior developers value metrics on learning progression and code quality improvement. Managers need ROI calculations comparing time saved versus tool costs, along with quality trend analysis over time.

Q10: What concerns would you have about relying on AI for code quality assurance?

Summary: Over-reliance emerged as the primary concern—developers might stop critically thinking if they trust the AI too much. Senior developers worried about model biases perpetuating bad patterns if trained on flawed codebases. Junior developers expressed concern about not understanding the reasoning behind AI suggestions, potentially stunting their growth. Managers worried about liability issues if AI-approved code causes security breaches. All stakeholders emphasized that human oversight must remain mandatory, with AI serving as an assistant rather than replacement for human reviewers.

2.2 Software Requirements

The software requirements for IntelliCode Review are organized into functional and non-functional categories, presented in Tables 1, 2, 3 and Table 4, respectively. Requirements are prioritized across three levels based on implementation timeline:

- **Level 1:** Must be implemented by December 2025 (core system features).
- **Level 2:** Planned for implementation by the semester end, depending on available time.
- **Level 3:** Represent future work extending beyond the current semester.

This prioritization strategy, following the methodology of Bacchelli and Bird [9], supports a realistic project scope while preserving a clear long-term vision for system evolution and feature expansion.

2.2.1 Functional Requirements

Level 1 – Will Implement (Guaranteed by December 2025)

Table 1: Level 1 Functional Requirements – Core Implementation

ID	Priority	Description
FR1	Level 1	The system shall accept code submissions in Python through file upload or GitHub repository integration.
FR2	Level 1	The system shall perform static code analysis using established tools (e.g., Pylint, Bandit) and consolidate results.
FR3	Level 1	The system shall detect and categorize common code smells (e.g., long methods, duplicate code, high complexity).
FR4	Level 1	The system shall identify basic security vulnerabilities following OWASP Top 10 guidelines.
FR5	Level 1	The system shall generate detailed issue reports with severity levels (Critical, High, Medium, Low).
FR6	Level 1	The system shall provide user authentication and role-based access for Developers and Reviewers.
FR7	Level 1	The system shall allow users to view and download review results through a web interface.
FR8	Level 1	The system shall maintain a history of completed reviews and user feedback.
FR9	Level 1	The system shall integrate with GitHub using OAuth and webhooks for automated review triggering.

Level 2 – Might Implement (Time Permitting)

Table 2: Level 2 Functional Requirements – Extended Features

ID	Priority	Description
FR10	Level 2	The system shall utilize a transformer-based model (e.g., CodeBERT) to provide intelligent code suggestions.
FR11	Level 2	The system shall generate automated refactoring suggestions with side-by-side code examples.
FR12	Level 2	The system shall support GitLab integration and Slack notifications for review events.
FR13	Level 2	The system shall display project-level dashboards showing code quality trends and issue frequencies.
FR14	Level 2	The system shall allow reviewers to mark AI suggestions as accepted, rejected, or false positive.
FR15	Level 2	The system shall allow team leads to configure rule sets for coding standards.

Level 3 – Future Work (Not Implementing This Semester)

Table 3: Level 3 Functional Requirements – Full Application Vision

ID	Priority	Description
FR16	Level 3	The system shall support additional languages such as Java, JavaScript, and Go.
FR17	Level 3	The system shall implement continuous learning from reviewer feedback to improve AI accuracy.
FR18	Level 3	The system shall provide IDE plugins for pre-commit code review (VS Code, IntelliJ).
FR19	Level 3	The system shall support RESTful API access for integration with CI/CD pipelines.
FR20	Level 3	The system shall offer multilingual user interfaces (English, Spanish, Mandarin).
FR21	Level 3	The system shall predict defect probability using code metrics and historical review data.

2.2.2 Non-Functional Requirements

Table 4: Non-Functional Requirements for IntelliCode Review

ID	Priority	Description
NFR1	Level 1	The system shall analyze code files up to 1000 lines within 3 seconds for at least 90% of submissions under normal load conditions.
NFR2	Level 1	The platform shall support at least 50 concurrent users without significant performance degradation.
NFR3	Level 1	All data transmission shall use TLS 1.3 encryption, and all stored passwords shall be hashed using bcrypt with at least 12 rounds.
NFR4	Level 1	The web interface shall comply with WCAG 2.1 AA accessibility standards, supporting keyboard navigation and screen readers.
NFR5	Level 1	The system shall maintain a false positive rate below 20% for AI-generated issue detections, validated by expert reviewers.
NFR6	Level 2	The system shall perform automatic database backups every 6 hours and allow point-in-time recovery.
NFR7	Level 2	The deployment shall be containerized using Docker and deployable on both cloud and on-premise infrastructure.
NFR8	Level 3	The system shall provide real-time monitoring and performance dashboards using Prometheus and Grafana.

3 Use Case Modeling

3.1 Use Case Diagram

The use case diagram for IntelliCode Review is presented in Figure 2, illustrating the primary interactions between system actors and the platform's core functionalities. The system supports three primary actor types: Developers who submit code for review, Code Reviewers who evaluate submissions and AI suggestions, and System Administrators who manage platform configuration and users. Additionally, the GitHub Integration acts as an external system actor that triggers automated reviews through webhook events.

The diagram shows several key relationships. The "Submit Code for Review" use case includes the "Authenticate User" and "Run Static Analysis" use cases, as authentication and analysis are mandatory prerequisites for submission. The "View Review Results" use case extends to "Export Report," indicating that report export is an optional enhancement. Similarly, "Configure Rule Sets" includes "Validate Configuration" to ensure system integrity.

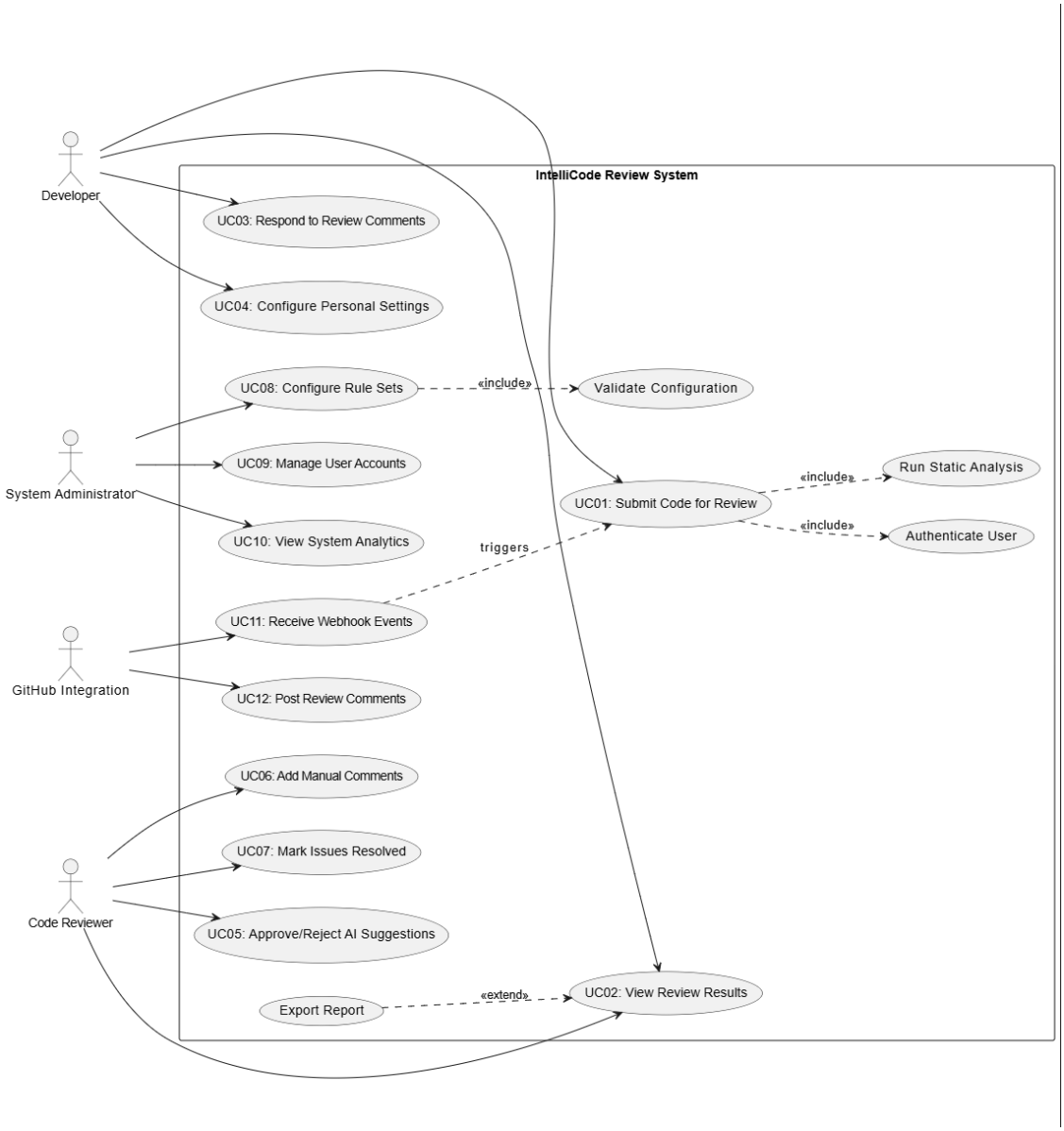

Figure 2: Use Case Diagram for IntelliCode Review System

As shown in Figure 2, the system architecture supports clear separation of concerns with distinct actor roles and well-defined use case boundaries.

3.2 Detailed Use Cases

Table 5 provides concise descriptions of all twelve use cases identified for IntelliCode Review. These use cases encompass the complete workflow from code submission through review completion, including both automated and human-driven processes. The functional requirements outlined in Table 1 and 2 map directly to these use cases, ensuring traceability between system requirements and user interactions.

Table 5: Detailed Use Cases for IntelliCode Review

ID	Use Case	Description
UC01	Submit Code for Review	Developer uploads code file or connects repository to initiate automated review process. System validates file format, checks authentication, queues analysis job with priority level, and returns submission confirmation with estimated completion time and tracking identifier.
UC02	View Review Results	Developer accesses comprehensive review report displaying categorized issues with severity ratings, line numbers, and explanatory comments. Results presented in collapsible sections organized by issue type (security, style, code smells) with filtering and sorting capabilities.
UC03	Respond to Review Comments	Developer reads AI-generated or human reviewer comments, marks individual issues as resolved with optional explanatory notes, requests additional clarification, or disputes findings. System updates issue status, sends notifications to relevant stakeholders, and maintains comment thread history.
UC04	Configure Personal Settings	User customizes notification preferences (email, Slack, in-app), sets default repository connection settings, selects preferred programming language for examples, and adjusts dashboard layout. System validates configuration against organizational policies and persists changes immediately with confirmation.
UC05	Approve or Reject AI Suggestions	Code reviewer evaluates each AI-identified issue, marks suggestions as accurate or false positive, adjusts severity ratings if necessary, and provides optional reasoning or additional context. System records feedback for machine learning model refinement and calculates reviewer agreement metrics.
UC06	Add Manual Comments	Reviewer supplements automated analysis with human insights on architecture decisions, business logic correctness, or nuanced quality issues. Comments support markdown formatting, code snippets, and file attachments. System associates comments with specific code lines and notifies the developer.
UC07	Mark Issues Resolved	After developer addresses flagged issues through code updates, reviewer verifies corrections meet standards, confirms issue resolution, and updates status. System tracks resolution time metrics, closes associated discussion threads, and updates project quality dashboard with resolved issue counts.

Continued on next page

Table 5 – continued from previous page

ID	Use Case	Description
UC08	Configure Rule Sets	Administrator creates new or modifies existing team-specific coding standards by enabling or disabling individual rules, adjusting severity level thresholds (e.g., maximum cyclomatic complexity), and adding custom pattern matching rules. System validates configuration syntax, performs compatibility checks, and applies changes to subsequent reviews.
UC09	Manage User Accounts	Administrator creates new user accounts with role assignment (Developer, Reviewer, Admin), manages access permissions for repositories and projects, resets passwords, and deactivates accounts. System enforces role-based access control policies and logs all administrative actions for audit purposes.
UC10	View System Analytics	Administrator accesses comprehensive dashboard displaying code quality trends over time, distribution of common issue types, average review velocity metrics, false positive rates by category, and user engagement statistics. System provides data export capabilities and customizable date range filtering.
UC11	Receive Webhook Events	System automatically receives pull request events from GitHub (opened, updated, closed), extracts changed files and metadata, validates webhook signature for security, triggers analysis pipeline with appropriate priority, and queues review job without requiring manual intervention.
UC12	Post Review Comments	System programmatically posts AI-generated review comments directly to GitHub pull requests with appropriate markdown formatting, severity indicator badges, line annotations, and explanation text. Comments are visible to all repository collaborators and integrate with GitHub's review conversation threads.

3.2.1 Expanded Use Case Templates

The following sections provide detailed templates for three critical use cases, documenting preconditions, flow of events, and postconditions. These templates follow the format presented in Tables 6, 7, and 8, providing comprehensive documentation for key system interactions.

Use Case: Submit Code for Review	
ID	UC01
Actor(s)	Developer, System
Precondition(s)	1. User has valid authentication credentials and active account. 2. User has code files ready for review in supported formats (Python, JavaScript, Java). 3. System has available processing capacity.
Flow of Events	1. Use case begins when developer navigates to submission interface. 2. Developer selects submission method (file upload or repository connection). 3. Developer chooses files or repository branch for review. 4. System validates file format and size constraints. 5. System authenticates user credentials and verifies permissions. 6. System creates review job with unique identifier. 7. System queues job in analysis pipeline based on priority. 8. System returns confirmation with job ID and estimated completion time. 9. System sends notification when analysis completes.
Postcondition(s)	1. Review job exists in system database with pending status. 2. Analysis pipeline has queued job for processing. 3. Developer has received job identifier for tracking.

Table 6: Submit Code for Review Use Case Template

Use Case: Approve or Reject AI Suggestions	
ID	UC05
Actor(s)	Code Reviewer, System
Precondition(s)	1. Code review has been completed by automated analysis system. 2. Review contains AI-generated suggestions requiring human validation. 3. Reviewer has appropriate permissions for the project.
Flow of Events	1. Use case begins when reviewer opens completed review report. 2. System displays all AI-identified issues with severity ratings. 3. Reviewer examines each suggestion with supporting evidence. 4. Reviewer marks suggestion as accepted, rejected, or false positive. 5. Reviewer optionally adjusts severity level with justification. 6. Reviewer provides optional feedback explaining decision. 7. System records feedback with timestamp and reviewer identity. 8. System updates issue status and calculates accuracy metrics. 9. System queues feedback for machine learning model retraining. 10. Reviewer repeats steps 3-9 for remaining suggestions. 11. Reviewer submits final review decision. 12. System notifies developer of review completion.
Postcondition(s)	1. All AI suggestions have reviewer validation status. 2. Feedback data stored for model improvement. 3. Developer notified of review completion with actionable items. 4. System metrics updated with reviewer agreement statistics.

Table 7: Approve or Reject AI Suggestions Use Case Template

Use Case: Configure Rule Sets	
ID	UC08
Actor(s)	System Administrator
Precondition(s)	1. Administrator has valid credentials with configuration permissions. 2. System has default rule set templates available. 3. No active reviews are using rules being modified.
Flow of Events	1. Use case begins when administrator navigates to rule set configuration interface. 2. Administrator selects existing rule set or creates new one. 3. Administrator views categorized list of available rules. 4. Administrator enables or disables specific rules by category. 5. Administrator adjusts severity thresholds for complexity metrics. 6. Administrator defines custom pattern matching rules using regex. 7. Administrator assigns rule set to specific teams or projects. 8. System validates configuration syntax and completeness. 9. System performs compatibility checks with existing reviews. 10. Administrator previews changes with sample code analysis. 11. Administrator saves configuration with descriptive name. 12. System applies new rules to subsequent review submissions. 13. System logs configuration change with timestamp.
Postcondition(s)	1. New or updated rule set exists in system configuration. 2. Rule set assigned to appropriate teams or projects. 3. Future reviews will use updated rule set. 4. Configuration change logged for audit trail.

Table 8: Configure Rule Sets Use Case Template

4 Glossary of Terms

1. **API (Application Programming Interface)** A set of protocols, routines, and tools for building software applications that specifies how software components should interact, enabling communication between different systems.

2. **Bcrypt** A password hashing function based on the Blowfish cipher, designed to be computationally expensive to resist brute-force attacks by incorporating a configurable work factor.

3. **CI/CD (Continuous Integration/Continuous Deployment)** Software engineering practices where code changes are automatically built, tested, and deployed to production environments with minimal manual intervention, enabling rapid and reliable software delivery.

4. **Code Smell** A surface indication in source code that suggests a deeper problem in system design or implementation, such as long methods, duplicate code, or excessive complexity that may impede maintenance and evolution.

5. **CodeBERT** A transformer-based pre-trained model developed by Microsoft Research for understanding programming languages and natural language, trained on millions of code-documentation pairs from GitHub repositories to capture semantic relationships.

6. **Cyclomatic Complexity** A software metric that quantifies the number of linearly independent paths through program source code, calculated by constructing the control flow graph and measuring its complexity, used to assess testing difficulty and maintenance risk.

7. **False Positive** An issue or vulnerability incorrectly identified by automated analysis that upon expert review is determined to be benign, reducing user trust and tool effectiveness when occurring frequently.

8. **OAuth (Open Authorization)** An open standard authorization protocol that enables third-party applications to obtain limited access to user accounts on services like GitHub without exposing user passwords, using token-based authentication.

9. **OWASP Top 10** A regularly-updated authoritative report published by the Open Web Application Security Project identifying the ten most critical web application security risks, including injection attacks, broken authentication, and sensitive data exposure.

10. **Pull Request (PR)** A GitHub mechanism for proposing code changes where developers submit modifications for review and discussion before merging into the main codebase, facilitating collaborative code review and quality control.

11. **PyTorch** An open-source machine learning framework providing tensor computation with strong GPU acceleration and automatic differentiation, along with neural network building blocks for Python, widely used in deep learning research and production.

12. **RESTful API** An architectural style for designing networked applications based on Representational State Transfer principles, using standard HTTP methods (GET, POST, PUT, DELETE) and stateless communication for resource manipulation.

13. **Static Analysis** The examination of source code without executing it to identify potential errors, security vulnerabilities, code quality issues, and style violations through pattern matching, abstract interpretation, and symbolic execution techniques.

14. **Technical Debt** The implied cost of additional rework caused by choosing quick or easy solutions instead of better approaches that would take longer, accumulating over time and creating maintenance burden that impedes future development.

15. **Transformer Model** A neural network architecture introduced in 2017 that uses self-attention mechanisms to process sequential data in parallel, enabling language models to understand long-range dependencies and contextual relationships effectively.

16. **Webhook** An HTTP callback mechanism that allows applications to provide real-time event notifications by sending automated POST requests to configured URLs when specific events occur, enabling loose coupling between systems.

5 References

References

[1] SonarSource. (2024). *SonarQube: Continuous Code Quality and Security*. SonarSource S.A. Available at: `https://www.sonarqube.org`

[2] CodeClimate. (2024). *Quality and Maintainability for Your Entire Stack*. Code Climate Inc. Available at: `https://codeclimate.com`

[3] DeepCode. (2023). *AI-Powered Semantic Code Analysis*. Snyk Limited. Available at: `https://www.deepcode.ai`

[4] Chen, M., Tworek, J., Jun, H., Yuan, Q., Pinto, H. P. D. O., Kaplan, J., Edwards, H., Burda, Y., Joseph, N., Brockman, G., Ray, A., Puri, R., Krueger, G., Petrov, M., Khlaaf, H., Sastry, G., Mishkin, P., Chan, B., Gray, S., Ryder, N., Pavlov, M., Power, A., Kaiser, L., Bavarian, M., Winter, C., Tillet, P., Such, F. P., Cummings, D., Plappert, M., Chantzis, F., Barnes, E., Herbert-Voss, A., Guss, W. H., Nichol, A., Paino, A., Tezak, N., Tang, J., Babuschkin, I., Balaji, S., Jain, S., Saunders, W., Hesse, C., Carr, A. N., Leike, J., Achiam, J., Misra, V., Morikawa, E., Radford, A., Knight, M., Brundage, M., Murati, M., Mayer, K., Welinder, P., McGrew, B., Amodei, D., McCandlish, S., Sutskever, I., & Zaremba, W. (2021). Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*.

[5] Codacy. (2024). *Automated Code Review and Quality Analysis Platform*. Codacy Inc. Available at: `https://www.codacy.com`

[6] McBurney, P. W., & McMillan, C. (2016). Automatic Source Code Summarization of Context for Java Methods. *IEEE Transactions on Software Engineering*, 42(2), 103–119. doi:10.1109/TSE.2015.2465386

[7] Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., & Zhou, M. (2020). CodeBERT: A Pre-Trained Model for Programming and Natural Languages. *Findings of the Association for Computational Linguistics: EMNLP 2020*, 1536–1547. doi:10.18653/v1/2020.findings-emnlp.139

[8] Allamanis, M., Barr, E. T., Devanbu, P., & Sutton, C. (2018). A Survey of Machine Learning for Big Code and Naturalness. *ACM Computing Surveys*, 51(4), 1–37. doi:10.1145/3212695

[9] Bacchelli, A., & Bird, C. (2013). Expectations, Outcomes, and Challenges of Modern Code Review. *Proceedings of the 2013 International Conference on Software Engineering (ICSE)*, 712–721. IEEE Press.

[10] OWASP Foundation. (2021). *OWASP Top Ten 2021: The Ten Most Critical Web Application Security Risks*. Available at: `https://owasp.org/Top10/`

[11] Fowler, M. (2018). *Refactoring: Improving the Design of Existing Code* (2nd ed.). Addison-Wesley Professional. ISBN: 978-0134757599.